

SNES hardware summary

A from-the-ground orientation to the Super Nintendo / Super Famicom for codegen and demo work in this repo. Companion references: the complete [register map](#) (generated from `platforms/snes/snes_*.h`) and the [65816 CPU reference](#). Facts are per the sources in [SOURCES.md](#) (nocash *fullsnes*, the SNESdev wiki, anomie's docs, Copetti).

At a glance

Part	What
CPU	Ricoh 5A22 — a WDC 65816 core (16-bit 6502 descendant) + DMA, hardware multiply/divide, joypad auto-read
CPU clock	3.58 MHz fast (FastROM / internal regs), 2.68 MHz slow (ROM default), 2.68/1.79 MHz for RAM/slow regions (NTSC)
Address space	24-bit, 16 MB: 256 banks × 64 KB
WRAM	128 KB in banks <code>\$7E – \$7F</code> ; the low 8 KB mirrored at <code>\$0000 – \$1FFF</code> in banks <code>\$00 – \$3F / \$80 – \$BF</code>
Video	dual PPU1 + PPU2 ; 64 KB VRAM, 512 B CGRAM (256 colours), 544 B OAM (128 sprites)
Audio	SPC700 CPU + S-DSP (8 voices) + 64 KB audio RAM — a separate processor behind 4 mailbox ports
Output	NTSC 256×224 @ ~60 Hz (or 512×448 hi-res/interlace); PAL 256×239 @ 50 Hz

CPU and memory

The 65816 boots in **6502 emulation mode** and is switched to **native mode** once at startup (`crt0` does `CLC`; `XCE`). In native mode the accumulator and index registers are independently 8- or 16-bit (the M and X status bits); `+mos-a16` runs with a 16-bit accumulator. See the [65816 reference](#) for the programming model and instruction set.

bank	contents of each 64 KB bank (offset \$0000 → \$FFFF)	
\$00–\$3F	→	low-RAM mirror (\$0000–\$1FFF) · I/O (\$2100/\$4200/\$4300) · LoROM \$8000+
\$40–\$7D	→	cartridge ROM
\$7E–\$7F	→	128 KB Work RAM (WRAM) ← high WRAM via far ptr or the \$2180 port
\$80–\$BF	→	mirror of \$00–\$3F (FastROM-capable)
\$C0–\$FF	→	cartridge ROM (HiROM)

The 24-bit address space is organised as **256 banks of 64 KB**. The system banks (`$00 – $3F`) and their fast mirror (`$80 – $BF`) share a common low layout:

Range (in a system bank)	Contents
<code>\$0000 – \$1FFF</code>	Low-RAM — a mirror of the first 8 KB of WRAM (<code>\$7E:0000</code>)
<code>\$2100 – \$213F</code>	PPU registers
<code>\$2140 – \$2143</code>	APU I/O ports
<code>\$2180 – \$2183</code>	WRAM access port
<code>\$4016 – \$4017</code>	Serial joypad
<code>\$4200 – \$421F</code>	CPU I/O (interrupts, mul/div, timers, auto-joypad)
<code>\$4300 – \$437F</code>	DMA / HDMA channels
<code>\$8000 – \$FFFF</code>	Cartridge ROM

Banks `$7E – $7F` are the full **128 KB of WRAM**. Cartridge ROM is mapped by the board: **LoROM** exposes 32 KB per bank at `$8000 – $FFFF` ; **HiROM** exposes a full 64 KB per bank. This repo's SNES platform is **LoROM** (see `platforms/snes/link.ld`). High WRAM (`$7E2000 +` and bank `$7F`) is reachable from a `$00` -bank program either through the WRAM port (`$2180 – $2183`) or with a 65816 far pointer (the `+mos-a16` far path).

A **FastROM** board + `MEMSEL` (`$420D`) bit 0 run banks `$80 +` at 3.58 MHz instead of 2.68 MHz.

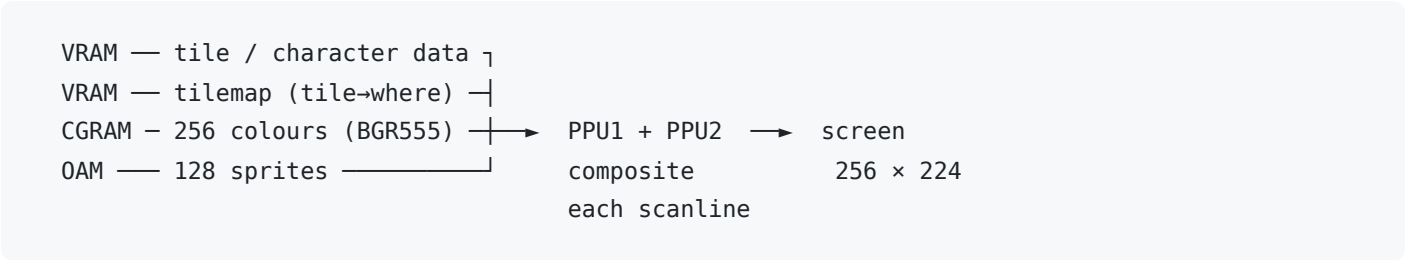
Video (PPU)

The PPU has **no linear framebuffer**. You upload three things into its private memories and it composites them every scanline:

- **VRAM** — 64 KB, **word-addressed** (32 K 16-bit words). Holds tile (character) data and tilemaps. Accessed through `VMADD / VMDATA` (`$2116` – `$2119`) with an auto-increment set by `VMAIN` .
- **CGRAM** — 256 palette entries, **BGR555** (`0bbbbbggggrrrrr` , 15-bit colour). Entry 0 is the backdrop, and pixel value 0 in any tile is transparent.
- **OAM** — 544 B: 128 sprites × 4 B (X, Y, tile, attributes) plus a 32 B high table (X bit 8 + size bit per sprite).

The access-window rule (the #1 "nothing shows / garbage shows" bug): VRAM, CGRAM and OAM are writable **only during force-blank or v-blank**. Writes during active display are dropped. Bring the machine up force-blanked (`INIDISP` bit 7), upload everything, then release the blank last. Also initialise **all** PPU control registers, not just the ones you use — power-on state is indeterminate and some emulators randomise it (`snes_ppu_reset_blank()` in `snes_ppu.h` does this).

How those memories combine into a frame:



Background modes

`BGMODE` (`$2105`) low three bits select the layer layout / bit depth:

Mode	Layers
0	4 × 2bpp (4-colour) BGs
1	BG1/BG2 4bpp (16-colour) + BG3 2bpp — the common one
2	BG1/BG2 4bpp with per-tile offset
3	BG1 8bpp (256-colour) + BG2 4bpp
4	BG1 8bpp + BG2 2bpp, per-tile offset
5	BG1 4bpp + BG2 2bpp, hi-res 512
6	BG1 4bpp hi-res, per-tile offset

Mode	Layers
7	one 8bpp layer with a full affine transform (rotate/scale; pseudo-3D via per-scanline HDMA of the matrix)

bpp = bits/pixel = colours per tile (2bpp = 4, 4bpp = 16, 8bpp = 256). Tiles are 8×8; normal modes store them as **bit-planes interleaved by row pair** (the fiddly part). Mode 7 is the exception — its character data is **linear, 1 byte/pixel** (no bit-planes), at the cost of a 256-tile cap and even/odd VRAM interleaving (tilemap in even bytes, chr in odd). Mode 7 is the natural fit for a per-pixel framebuffer-style image.

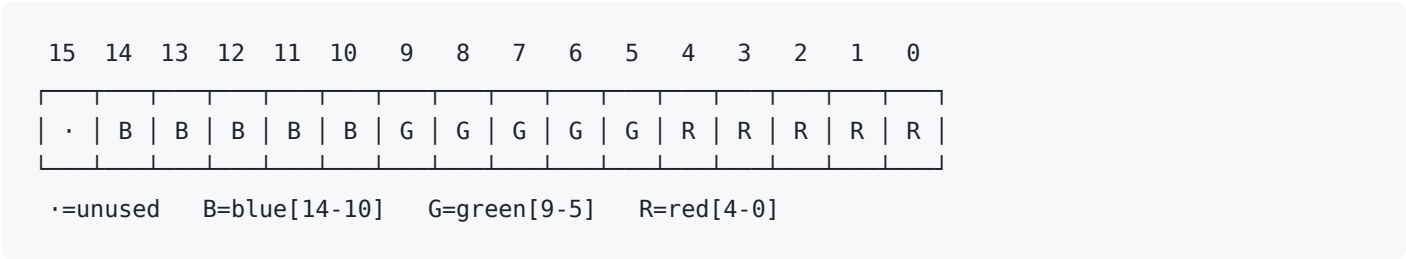
Tile (character) data format — 4bpp 8×8 tile = 32 bytes; bit-planes interleaved by row pair, MSB = leftmost pixel:

```

offset 0 : row0 plane0 | row0 plane1  ↵ planes 0+1, rows 0-7  = 16 B
offset 2 : row1 plane0 | row1 plane1  |
...                                     ↳
offset 16 : row0 plane2 | row0 plane3  ↵ planes 2+3, rows 0-7  = 16 B
...                                     ↳
a pixel's 4-bit index = (p3 p2 p1 p0) read down the planes at that column
2bpp = 16 B (planes 0+1) · 8bpp = 64 B (planes 0-7) · Mode 7 chr = linear 1 byte/pixel

```

Colour format — each CGRAM entry is 16-bit **BGR555** (bit 15 ignored); `SNES_RGB(r,g,b)` packs it:

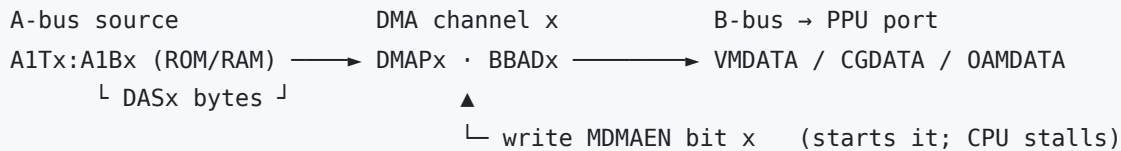


Sprites, windows, colour math

Up to **128 sprites**, ~32 per scanline, sizes 8×8 to 64×64 (a pair selectable via `OBSEL`). Two **window** regions can mask any layer (`W12SEL` / `W34SEL` / `W0BJSEL` , `WH0` — `WH3`), and a **colour-math** unit adds/subtracts a fixed colour or the sub-screen (`CGWSEL` / `CGADSUB` / `COLDATA`) for transparency and lighting effects.

DMA / HDMA

Eight channels (`$4300 – $437F`). **General-purpose DMA** (triggered by `MDMAEN` `$420B`) blasts up to 64 KB CPU → PPU (or back) while the CPU stalls — the only practical way to fill VRAM in one v-blank. **HDMA** (`HDMAEN` `$420C`) streams a few bytes to registers each scanline, for gradients, window animation, and Mode-7 matrix updates. Same access-window rule applies: do GP-DMA into VRAM/CGRAM/OAM in force-blank or v-blank. See the DMA / HDMA section of the [register map](#).



Audio

The audio subsystem is a **physically separate computer**: an **SPC700** CPU with its own 64 KB RAM and an **S-DSP** (8 stereo voices, ADSR, echo). The 65816 cannot address any of it directly — all communication is a handshake through the four **APU I/O ports** (`$2140 – $2143`) with the SPC700 boot ROM. Its internal registers are therefore out of scope for the CPU-visible register map.

Timing and interrupts

One NTSC frame is ~224 visible scanlines + ~38 of v-blank, ~1/60 s. The **v-blank** window is the per-frame budget for VRAM/CGRAM/OAM writes (a few KB by hand, far more via DMA). Enable the **v-blank NMI** with `NMITIMEN` (`$4200`) bit 7; an optional **H/V IRQ** fires at a programmed `HTIME / VTIME` position. The usual loop: compute into a RAM shadow during active display, then in the NMI handler DMA the changed bytes to the PPU during v-blank. A purely static image can instead just force-blank, build everything once, and release the blank — the simplest thing that displays.

```

one NTSC frame ≈ 262 scanlines ≈ 1/60 s
line 0 | █████ visible display (~224 lines) █████ VRAM/CGRAM/OAM writes: DROPPED
line 224 | — v-blank begins ————— NMI fires (NMITIMEN bit 7)
        | █████ v-blank (~38 lines) █████ VRAM/CGRAM/OAM + DMA: SAFE
line 262 | (next frame)
loop: compute into a RAM shadow during display → DMA it to the PPU during v-blank
  
```